

IVOR PAINE MEMORIAL HOSPITAL

Database Development — Milestone 2
Documentation

Group Members:

22i-0939 | 22i-0800 | 22i-2327 | 22i-1264

Contents

1. Drop Order — Why It Matters	3
2. Referential Integrity — Explained	3
3. Normalization Steps	4
3.1. First Normal Form (1NF)	4
3.2. Second Normal Form (2NF)	5
3.3. Third Normal Form (3NF)	6
3.4. Normalization Summary	7
4. Table Descriptions	8
5. DDL Constraint Summary	11
6. Initial Data Summary	12

1. Drop Order — Why It Matters

When dropping tables in SQL Server, child tables (those that hold FOREIGN KEY columns) must be dropped before parent tables (those being referenced). Dropping a parent while a child still references it raises error 3726: 'Could not drop object because it is referenced by a FOREIGN KEY constraint.'

The schema has two circular FK pairs that require special handling before dropping:

Circle 1 — nurse ↔ care_unit

nurse.care_unit_no → care_unit(care_unit_no) AND care_unit.in_charge_staff_no → nurse(staff_no). Fix: execute ALTER TABLE nurse DROP CONSTRAINT fk_nurse_careunit to break the circle, then drop care_unit, then nurse.

Circle 2 — doctor ↔ consultant

doctor.consultant_no → consultant(staff_no) AND consultant.staff_no → doctor(staff_no). Fix: drop the forward FK fk_doctor_consultant first, then drop consultant, then doctor.

Drop Order		
Step	Table / Action	Reason
1	performance_review	Leaf — nothing references it
2	doctor_experience	Leaf — nothing references it
3	patient_complaint_treatment	References patient, doctor, complaint, treatment
4	patient	References ward, care_unit, bed, doctor, consultant
5	bed	References ward
6	ALTER TABLE nurse DROP fk_nurse_careunit	Break nurse ↔ care_unit circle
7	care_unit	References ward, nurse (now safe)
8	nurse	References staff, ward
9	ALTER TABLE doctor DROP fk_doctor_consultant	Break doctor ↔ consultant circle
10	consultant	References doctor (now safe)
11	doctor	References staff
12	staff	Supertype — root of staff hierarchy
13	ward	References specialty
14	specialty	Root lookup table
15	complaint	Root lookup table
16	treatment	Root lookup table

2. Referential Integrity — Explained

Referential Integrity (RI) is the guarantee that every FOREIGN KEY value in a child table actually exists as a PRIMARY KEY (or unique key) value in the referenced parent table. It prevents orphan rows — for example, a patient row pointing to a ward_name that has been deleted.

2.1 What is a Foreign Key?

A FOREIGN KEY is a column (or set of columns) in one table (the child) whose values must match values that already exist in the PRIMARY KEY of another table (the parent). Examples from this schema:

- patient.ward_name → ward(ward_name)
- patient.care_unit_no → care_unit(care_unit_no)
- patient.in_charge_doctor_no → doctor(staff_no)

2.2 What SQL Server Does Automatically

Once a FOREIGN KEY constraint exists, SQL Server will:

- Reject INSERT / UPDATE that reference a non-existent parent key.
- Reject DELETE / UPDATE on a parent row that still has dependent children (unless ON DELETE CASCADE / SET NULL is specified).

2.3 How We Maintain It in This Schema

Referential Integrity Rules	
Rule	How We Implement It
Every patient belongs to a real ward	patient.ward_name FK → ward(ward_name)
Every patient has a real care unit	patient.care_unit_no FK → care_unit
Every care unit has a real staff nurse in charge	care_unit.in_charge_staff_no FK → nurse
Every nurse works on a real ward	nurse.ward_name FK → ward
A bed is used by at most one patient	UNIQUE(bed_no) on patient table
Every treatment record has a real patient, complaint, treatment, doctor	4 FKs on patient_complaint_treatment
Every performance review references a real doctor and consultant	2 FKs on performance_review
Circular FKs handled with deferred inserts	Insert NULLs first, UPDATE after both sides exist (consultant_no, care_unit_no)
Date logic enforced	CHECK constraints: discharged >= admitted, treatment ended >= started

Tip: Because nurse ↔ care_unit reference each other, we insert nurses with care_unit_no = NULL, then insert care_unit rows, then UPDATE nurses to set care_unit_no. This is the correct pattern for any circular dependency.

3. Normalization Steps

Normalization is the process of structuring a relational database to reduce data redundancy and improve data integrity. We apply normalization to the columns of each table — not to the data inserted into it. The goal is to ensure that every column in every table depends only on the primary key of that table, and nothing else.

We apply normalization up to Third Normal Form (3NF). Each normal form builds on the previous:

- 1NF — Eliminate repeating groups; every column must hold a single atomic value.
- 2NF — Eliminate partial dependencies; every non-key column must depend on the whole primary key.
- 3NF — Eliminate transitive dependencies; every non-key column must depend directly on the primary key, not on another non-key column.

3.1 First Normal Form (1NF)

Rule

Every column in every table must hold a single, atomic (indivisible) value. There must be no repeating groups — no column storing multiple values, and no pattern of columns like complaint1, complaint2, complaint3 representing the same type of information.

Problem Found in the Source Forms

The Patient Record form contains a repeating group for medical history:

Patient Record — Flat / Unnormalized Form (UNF)					
patient_no	patient_name	complaint_code	treatment_code	doctor_no	date_started
P001	Alice Thompson	C001, C006	T001, T005	D01	2025-01-06, 2025-01-25
P002	Brian Okafor	C002	T003	D02	2025-01-09

Patient P001 has two complaints and two treatments. Storing them as comma-separated values in a single column violates 1NF because the column is not atomic.

Similarly, the Consultant Team Record had repeating groups for Previous Experience (multiple past roles per doctor) and Progress Reviews (multiple reviews per doctor over time).

Fix Applied

Each repeating group is extracted into its own table with a proper primary key:

1NF — Tables Created to Eliminate Repeating Groups		
Repeating Group	Table Created	Primary Key
Complaint + Treatment per Patient	patient_complaint_treatment	(patient_no, complaint_code, treatment_code, date_started)
Previous Experience per Doctor	doctor_experience	experience_id (surrogate)

1NF — Tables Created to Eliminate Repeating Groups		
Repeating Group	Table Created	Primary Key
Progress Reviews per Doctor	performance_review	review_id (surrogate)

After 1NF — patient_complaint_treatment (each row is atomic)					
patient_no	complaint_code	treatment_code	date_started	doctor_no	date_ended
P001	C001	T001	2025-01-06	D01	NULL
P001	C006	T005	2025-01-25	D01	NULL
P002	C002	T003	2025-01-09	D02	2025-02-15

Every cell now holds exactly one value. 1NF is satisfied across all tables in the schema.

3.2 Second Normal Form (2NF)

Rule

A table is in 2NF if it is already in 1NF and every non-key column depends on the entire primary key — not just part of it. Partial dependency is only possible when a table has a composite primary key.

Problem Found

Consider the following hypothetical flat table combining ward and patient data:

Hypothetical Flat Table — WARD_PATIENT (before 2NF)					
ward_name (PK)	patient_no (PK)	specialty_name	patient_name	date_admitted	bed_no
Ward A	P001	Orthopedics	Alice Thompson	2025-01-05	1
Ward A	P002	Orthopedics	Brian Okafor	2025-01-08	2
Ward B	P006	Geriatrics	Frank Diallo	2025-02-03	9

specialty_name depends only on ward_name, not on the full composite key (ward_name, patient_no). This is a partial dependency — a 2NF violation. Consequence: if Ward A changes specialty, every patient row for Ward A must be updated — an update anomaly.

Fix Applied

Specialty information is moved to a dedicated SPECIALTY table and referenced from WARD via a foreign key. PATIENT then references WARD:

After 2NF — Decomposed Tables		
Table	Primary Key	Non-Key Columns and Their Dependency
specialty	specialty_id	specialty_name → depends on specialty_id (full PK)

After 2NF — Decomposed Tables		
Table	Primary Key	Non-Key Columns and Their Dependency
ward	ward_name	specialty_id → depends on ward_name (full PK)
patient	patient_no	ward_name, bed_no, date_admitted ... → all depend on patient_no (full PK)

2NF Check — patient_complaint_treatment (composite PK table)		
Column	Depends On	Verdict
doctor_no	Full PK: (patient_no, complaint_code, treatment_code, date_started)	No partial dependency
date_ended	Full PK: (patient_no, complaint_code, treatment_code, date_started)	No partial dependency

All tables in the schema are in 2NF.

3.3 Third Normal Form (3NF)

Rule

A table is in 3NF if it is already in 2NF and no non-key column depends on another non-key column. Such a dependency — where column A determines column B and neither is the primary key — is called a transitive dependency.

Problem Found

Consider the following hypothetical flat patient table that stores specialty information directly:

Hypothetical Flat Table — PATIENT with specialty_name (before 3NF)					
patient_no (PK)	patient_name	ward_name	specialty_name	in_charge_doctor_no	consultant_no
P001	Alice Thompson	Ward A	Orthopedics	D01	C11
P006	Frank Diallo	Ward B	Geriatrics	D05	C12
P011	Karen Adeyemi	Ward C	Cardiology	D04	C12

The dependency chain:

- patient_no → ward_name (a patient is assigned to a ward)
- ward_name → specialty_name (a ward has a specialty)
- Therefore: patient_no → ward_name → specialty_name

specialty_name is transitively dependent on patient_no through ward_name. It does not depend directly on the primary key — this is a 3NF violation.

Similarly, if consultant specialty were stored on every patient row instead of in the CONSULTANT table, it would be transitively dependent via consultant_no.

Fix Applied

Transitive dependencies are broken by placing each attribute in the table whose key it directly depends on:

After 3NF — Dependency Chain Resolved		
Table	Primary Key	What Its Non-Key Columns Directly Depend On
specialty	specialty_id	specialty_name depends directly on specialty_id
ward	ward_name	specialty_id (FK) depends directly on ward_name
patient	patient_no	ward_name (FK) depends directly on patient_no — NOT on specialty
consultant	staff_no	specialty depends directly on staff_no
patient	patient_no	consultant_no (FK) depends directly on patient_no

There is no attribute in any table that depends on a non-key column of the same table. All tables in the schema are in 3NF.

3.4 Normalization Summary

Complete Normalization Summary — 1NF through 3NF			
NF	Violation Identified	Table(s) Affected	Resolution
1NF	Repeating group: multiple complaints & treatments per patient on Patient Record form	patient_complaint_treatment	Extracted repeating group into separate table with composite PK (patient_no, complaint_code, treatment_code, date_started)
1NF	Repeating group: multiple previous experience entries per doctor on Consultant Team Record	doctor_experience	Extracted into doctor_experience with surrogate PK (experience_id)
1NF	Repeating group: multiple performance reviews per doctor over time	performance_review	Extracted into performance_review with surrogate PK (review_id)
2NF	Partial dependency: specialty_name depends only on ward_name, not on (ward_name, patient_no)	ward, specialty, patient	Moved specialty_name to specialty table; ward holds specialty_id FK; patient holds ward_name FK
2NF	No violation in patient_complaint_treatment: doctor_no and date_ended both depend on the full composite PK	patient_complaint_treatment	No change needed — already in 2NF

Complete Normalization Summary — 1NF through 3NF			
NF	Violation Identified	Table(s) Affected	Resolution
3NF	Transitive dependency: patient_no → ward_name → specialty_name	patient, ward, specialty	Resolved by keeping specialty only in specialty table; patient references ward, ward references specialty
3NF	Transitive dependency: consultant specialty would be repeated on every patient row via consultant_no	patient, consultant	Consultant specialty stored only in consultant table; patient holds consultant_no FK

Final Schema — All 14 Tables

After applying 1NF, 2NF, and 3NF, the final normalized schema consists of the following 14 tables. Each table is in 3NF: every non-key column depends on the primary key, the whole primary key, and nothing but the primary key.

Final Normalized Schema — 14 Tables		
Table	Primary Key	Purpose / Non-Key Columns Depend On PK Because...
specialty	specialty_id	Lookup: specialty_name directly names the specialty
ward	ward_name	specialty_id identifies the ward's medical focus
staff	staff_no	staff_name is the name of that specific staff member
doctor	staff_no (FK→staff)	position, date_joined_team, consultant_no all describe that doctor
consultant	staff_no (FK→doctor)	specialty describes that consultant's clinical field
nurse	staff_no (FK→staff)	nurse_type, ward_name, care_unit_no describe that nurse's role
care_unit	care_unit_no	ward_name and in_charge_staff_no describe that care unit
bed	bed_no	ward_name says which ward this bed belongs to
complaint	complaint_code	complaint_desc names the complaint for that code
treatment	treatment_code	treatment_desc names the treatment for that code
patient	patient_no	All patient details (name, DOB, dates, ward, bed, doctor) describe that patient
patient_complaint_treatment	(patient_no, complaint_code, treatment_code, date_started)	doctor_no and date_ended describe this specific treatment episode
doctor_experience	experience_id	from_date, to_date, position, establishment describe that experience record
performance_review	review_id	review_date, grade, reviewed_by_consultant describe that review

4. Table Descriptions

specialty

Lookup table for medical specialties (e.g., Orthopedics, Cardiology). Represents what a ward caters for. A specialty may be treated on more than one ward.

specialty			
Column	Type	Constraint	Description
specialty_id	INT	PK	Unique numeric identifier
specialty_name	VARCHAR(100)	UNIQUE NOT NULL	Human-readable specialty name

ward

Each ward has a unique name and caters for exactly one specialty. Multiple wards may share a specialty.

ward			
Column	Type	Constraint	Description
ward_name	VARCHAR(100)	PK	Unique ward identifier
specialty_id	INT	FK → specialty	Specialty treated on this ward

staff

Supertype table for all hospital staff. Both NURSE and DOCTOR are subtypes that inherit staff_no as their PK and FK. Stores shared attributes only.

staff			
Column	Type	Constraint	Description
staff_no	INT	PK	Unique staff number
staff_name	VARCHAR(100)	NOT NULL	Full name of the staff member

doctor

Subtype of STAFF. Every row in doctor has a matching row in staff. Holds role and team membership. A doctor belongs to at most one consultant's team.

doctor			
Column	Type	Constraint	Description
staff_no	INT	PK, FK → staff	Inherited from staff
position	VARCHAR(30)	NOT NULL, CHECK	Student Junior Houseman Senior Houseman Asst Registrar Registrar
date_joined_team	DATE	NOT NULL	When this doctor joined the current team

doctor			
Column	Type	Constraint	Description
consultant_no	INT	FK → consultant, NULL	The consultant leading this doctor's team; NULL for consultants themselves

consultant

Subtype of DOCTOR (partial specialisation). Only some doctors are consultants. Each consultant has a single specialty and leads a team of doctors.

consultant			
Column	Type	Constraint	Description
staff_no	INT	PK, FK → doctor	Inherited from doctor
specialty	VARCHAR(100)	NOT NULL	The clinical specialty of this consultant

nurse

Subtype of STAFF. Works on exactly one ward and one care unit. nurse_type determines role: Day Sister, Night Sister, Staff Nurse, Non-Registered.

nurse			
Column	Type	Constraint	Description
staff_no	INT	PK, FK → staff	Inherited from staff
nurse_type	VARCHAR(20)	NOT NULL, CHECK	Day Sister Night Sister Staff Nurse Non-Registered
ward_name	VARCHAR(100)	FK → ward	Ward this nurse works on
care_unit_no	INT	FK → care_unit, NULL	Care unit this nurse is assigned to

care_unit

A group of patients cared for by a dedicated group of nurses. Each care unit has a unique number hospital-wide and is located within one ward. A Staff Nurse is in charge of each care unit.

care_unit			
Column	Type	Constraint	Description
care_unit_no	INT	PK	Hospital-wide unique care unit number
ward_name	VARCHAR(100)	FK → ward	Ward containing this care unit
in_charge_staff_no	INT	FK → nurse	Staff nurse responsible for this unit

bed

Each bed has a hospital-wide unique number and belongs to one ward. Discharged patients release their bed (enforced by UNIQUE constraint on patient.bed_no).

bed			
Column	Type	Constraint	Description
bed_no	INT	PK	Hospital-wide unique bed number
ward_name	VARCHAR(100)	FK → ward	Ward in which this bed is located

complaint

Lookup table for medical complaint types (e.g., Hip Fracture, Asthma). Codes correspond to the Complaint Code on the Patient Record form.

complaint			
Column	Type	Constraint	Description
complaint_code	VARCHAR(20)	PK	Short code (e.g., C001)
complaint_desc	VARCHAR(255)	NOT NULL	Full description of the complaint

treatment

Lookup table for treatment types (e.g., Physiotherapy, Chemotherapy). Codes correspond to the Treatment Code on the Patient Record form.

treatment			
Column	Type	Constraint	Description
treatment_code	VARCHAR(20)	PK	Short code (e.g., T001)
treatment_desc	VARCHAR(255)	NOT NULL	Full description of the treatment

patient

Core patient entity. Combines information from the Patient Record and Ward Record forms. A patient occupies exactly one bed (UNIQUE constraint), belongs to one ward, one care unit, is under one in-charge doctor, and one consultant.

patient			
Column	Type	Constraint	Description
patient_no	INT	PK	Unique patient identifier
patient_name	VARCHAR(100)	NOT NULL	Full name
date_of_birth	DATE	NOT NULL	Date of birth
date_admitted	DATE	NOT NULL	Date admitted to hospital
date_discharged	DATE	NULL	NULL = currently admitted; CHECK >= date_admitted

patient			
Column	Type	Constraint	Description
ward_name	VARCHAR(100)	FK → ward	Assigned ward
care_unit_no	INT	FK → care_unit	Assigned care unit
bed_no	INT	FK → bed, UNIQUE	Occupied bed (one patient per bed)
in_charge_doctor_no	INT	FK → doctor	Primary responsible doctor
consultant_no	INT	FK → consultant	Overseeing consultant

patient_complaint_treatment

Ternary associative (weak) entity capturing the medical history. Each row represents one treatment episode for one complaint of one patient, administered by a specific doctor between two dates.

patient_complaint_treatment			
Column	Type	Constraint	Description
patient_no	INT	PK, FK → patient	Patient receiving treatment
complaint_code	VARCHAR(20)	PK, FK → complaint	Complaint being treated
treatment_code	VARCHAR(20)	PK, FK → treatment	Type of treatment given
date_started	DATE	PK	Start date of this treatment episode
doctor_no	INT	FK → doctor	Doctor administering the treatment
date_ended	DATE	NULL	NULL = treatment ongoing; CHECK >= date_started

doctor_experience

Multi-valued attribute from the Consultant Team Record (Previous Experience section). Records past positions held by a doctor at various establishments.

doctor_experience			
Column	Type	Constraint	Description
experience_id	INT	PK	Surrogate key
staff_no	INT	FK → doctor	Doctor this experience belongs to
from_date	DATE	NOT NULL	Start date of this role
to_date	DATE	NULL	NULL = current role; CHECK >= from_date
position	VARCHAR(100)	NOT NULL	Job title held
establishment	VARCHAR(255)	NOT NULL	Hospital or institution name

performance_review

Multi-valued attribute from the Consultant Team Record (Progress section). A consultant assigns a performance grade to each doctor in their team every six months.

performance_review			
Column	Type	Constraint	Description
review_id	INT	PK	Surrogate key
doctor_no	INT	FK → doctor	Doctor being reviewed
review_date	DATE	NOT NULL	Date of the review
performance_grade	VARCHAR(10)	NOT NULL	Grade assigned (e.g., A, B+)
reviewed_by_consultant	INT	FK → consultant	Consultant who assigned the grade

5. DDL Constraint Summary

The table below lists every integrity constraint applied in the schema beyond basic PRIMARY KEY and FOREIGN KEY declarations.

Key Constraints			
Constraint Name	Table	Definition	Business Rule
ck_doctor_position	doctor	position IN ('Student','Junior Houseman','Senior Houseman','Asst Registrar','Registrar')	Only valid positions permitted
ck_nurse_type	nurse	nurse_type IN ('Day Sister','Night Sister','Staff Nurse','Non-Registered')	Only valid nurse categories
ck_patient_dates	patient	date_discharged IS NULL OR date_discharged >= date_admitted	Cannot be discharged before admitted
uq_patient_bed	patient	UNIQUE(bed_no)	One patient per bed at any time
ck_pct_dates	patient_complaint_treatment	date_ended IS NULL OR date_ended >= date_started	Treatment cannot end before it starts
ck_experience_dates	doctor_experience	to_date IS NULL OR to_date >= from_date	Experience end >= start

6. Initial Data Summary

The attached SQL script populates the database with realistic sample data meeting all minimum requirements.

Record Counts per Table		
Table	Count	Notes
specialty	10	Orthopedics, Geriatrics, Cardiology, Neurology, Oncology, Pediatrics, Dermatology, Pulmonology, Gastroenterology, Endocrinology

Record Counts per Table		
Table	Count	Notes
ward	10	Ward A–J, each mapped to one specialty
staff	41	14 doctors/consultants + 27 nurses
doctor	14	10 junior/mid doctors + 4 consultants; positions cover all CHECK values
consultant	4	Blake (Ortho), Thornton (Cardio), Murphy (Neuro), Zahra (Oncology)
nurse	27	Covers all 4 nurse_type values; spread correctly across all 8 active wards
care_unit	8	One per active ward (A–H); each with a Staff Nurse from the same ward in charge
bed	50	Distributed across all 10 wards
complaint	15	Covers wide range: fractures, cardiac, neurological, oncological, pulmonary
treatment	15	Surgical, pharmaceutical, and therapeutic treatments
patient	30	Ages from ~30 to ~90; dates span Jan–Apr 2025; mix of current and discharged
patient_complaint_treatment	38	Many patients have multiple complaints; several have multiple treatment episodes
doctor_experience	20	Each doctor has 1–3 prior roles across various hospitals
performance_review	20	Every doctor reviewed twice (6-month interval) by their consultant

Referential integrity verification: All INSERT statements are ordered so that parent rows exist before child rows reference them. Circular dependencies (nurse ↔ care_unit and doctor ↔ consultant) are resolved by inserting NULL initially and using UPDATE statements after both sides exist.